

Probabilistic ResNet with Self-Attention for Adversarial LoRa Fingerprinting

Harish Kumar N P and Dudyala Sai Ganesh

230102038

230102028

Course Code: EE337

Abstract

Deep learning classifiers for LoRa Radio Frequency Fingerprinting (RFFI) fail against zero-day spoofing attacks by forcefully mapping unknown signals to known device classes. We implement a Probabilistic Multi-Task ResNet with Self-Attention, using Monte Carlo (MC) Dropout ($T = 10$) for Bayesian uncertainty estimation. A baseline MC Dropout shield detects only 30% of rogue signals; 70% evade detection due to overlapping latent clusters. We address this via FGSM adversarial fine-tuning ($\epsilon = 0.05$) to restructure the feature space, combined with a 1-Nearest Neighbour (1-NN) anomaly detector on 64-dim feature vectors that bypasses the softmax head entirely. Experiments confirm clear separation between legitimate and rogue signals at threshold $\tau = 0.80$, validated by t-SNE and distance histogram visualisations.

I. INTRODUCTION

A. Brief Introduction to the Research Topic

The rapid proliferation of IoT devices has made device authentication a critical requirement. LoRaWAN is widely deployed for long-range, low-power IoT communication, but its MAC-layer authentication relies on pre-shared keys and software addresses that can be trivially cloned by an adversary with a software-defined radio (SDR) [1].

Radio Frequency Fingerprint Identification (RFFI) offers a physical-layer defence. Every transmitter carries unavoidable manufacturing imperfections—Carrier Frequency Offset (CFO), I/Q imbalance, oscillator phase noise—that distort its waveform in a device-specific, reproducible manner, embedding a unique “hardware biometric” into every packet. A receiver extracts these fingerprints to authenticate the transmitter without any modification to the transmitter hardware [1].

Deep learning has transformed RFFI by learning discriminative representations directly from raw I/Q samples, achieving high accuracy on known device sets. However, the softmax output layer forcefully assigns every input—including signals from unseen rogue transmitters—to a known class with artificially high confidence [2]. A rogue device spoofing a legitimate MAC address is thus misclassified as a known device, a zero-day breach that closed-set accuracy optimisation cannot resolve. A further challenge is adversarial vulnerability: imperceptible I/Q perturbations can dramatically degrade classifier accuracy [2]. This work addresses both flaws through Bayesian uncertainty estimation, adversarial latent-space hardening, and feature-space anomaly detection.

B. Related Works

Scalable LoRa RF Fingerprinting [1]. Shen et al. propose a scalable RFFI framework that replaces the softmax classifier with a deep metric learning-powered Radio Frequency Fingerprint (RFF) extractor trained using triplet loss. The extractor maps LoRa I/Q packets to a 512-dimensional embedding space where same-device signals cluster tightly and different-device signals are separated, generalising to previously unseen devices without retraining. A channel-independent spectrogram—constructed by dividing adjacent STFT columns—mitigates wireless channel effects, and data augmentation with simulated multipath and Doppler distortions improves robustness. Evaluation on 60 commercial LoRa devices achieves 98.40% accuracy on unseen devices and an ROC AUC of 0.9905 for rogue detection.

This work directly informs our ResNet backbone, global average pooling to a compact feature vector, and 1-NN anomaly detection. The key limitation is that the latent space targets channel robustness only and is not hardened against adversarial perturbations, leaving it vulnerable to the attacks in [2].

Adversarial Attack and Defense for LoRa [2]. Sagduyu and Erpek evaluate the adversarial vulnerability of deep learning-based LoRa authentication across device identification and rogue signal detection tasks, using single-task and multi-task CNN and FNN classifiers on over-the-air I/Q data. FGSM attacks dramatically reduce accuracy at low perturbation-to-signal ratios, and a hybrid gradient attack achieves high success against both tasks simultaneously. Adversarial training restores clean-data accuracy while reducing attack success probability from 0.987 to 0.003 (CNN). We repurpose this finding: rather than using adversarial training to restore accuracy,

we use it as a geometric regulariser to force tighter latent clusters that push rogue signals outside all known device regions.

Research Gap. Shen et al. address channel robustness but not adversarial perturbations. Sagduyu and Erpek address adversarial defence of closed-set classifiers but not open-set zero-day detection. Neither combines adversarial training with feature-space geometry restructuring for nearest-neighbour anomaly detection—the novel contribution here.

II. PROOF OF CONCEPT

A. System Model

The input is a 16,384-point LoRa I/Q array reshaped into a 128×128 single-channel image for 2D convolution. The ProbabilisticMultiTaskResNet consists of: (1) a 7×7 stem convolution (32 channels, stride 2, BatchNorm, ReLU); (2) three ResNetBlock layers (32, 64, 64 channels) with residual skip connections;

$$\text{out} = F(\mathbf{x}, \{W_i\}) + \mathbf{x}; \quad (1)$$

(3) a channel-wise Self-Attention module; (4) global average pooling to a 64-dim feature vector; and (5) MC Dropout ($p = 0.3$) feeding two linear heads—device identification (30 classes) and binary rogue detection. The full pipeline is illustrated in Fig. 1.

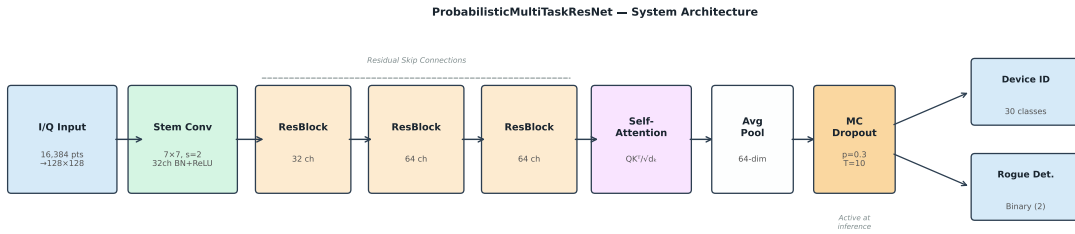


Fig. 1. Architecture of the ProbabilisticMultiTaskResNet. Raw LoRa I/Q data is reshaped and processed through a stem convolution, three residual blocks with skip connections, a Self-Attention module, global average pooling, and MC Dropout before branching into the device identification and rogue detection heads.

B. Analysis

Self-Attention. Given query Q , key K , and value V projections of the feature map:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad (2)$$

blended with the input via a learned scalar γ (initialised to zero): $\mathbf{out} = \gamma \cdot \text{Attention}(Q, K, V) + \mathbf{x}$. Scaling by $\sqrt{d_k}$ prevents dot-product saturation and stabilises gradients.

MC Dropout Uncertainty. With dropout active at inference, $T = 10$ stochastic forward passes yield $\{\hat{p}_t\}_{t=1}^T$. Epistemic uncertainty is measured by predictive variance:

$$\bar{p} = \frac{1}{T} \sum_{t=1}^T \hat{p}_t, \quad \sigma^2 = \frac{1}{T} \sum_{t=1}^T (\hat{p}_t - \bar{p})^2. \quad (3)$$

Signals with $\sigma^2 > 0.05$ are flagged as anomalous.

C. Results and Discussions

Training used 15,000 samples, Adam ($lr = 10^{-3}$), batch size 32, and 30 epochs with joint cross-entropy loss $\mathcal{L} = \mathcal{L}_{\text{device}} + \mathcal{L}_{\text{rogue}}$. Device classification converged at $\approx 50.67\%$ —an expected consequence of the heavy dropout regularisation required for meaningful Bayesian variance during inference.

On 10 unseen rogue signals, MC Dropout detected 30% (variance spikes up to $\sigma^2 = 0.177$), but 70% evaded the threshold ($\sigma^2 \approx 0.002$). This confirms the core limitation: *probabilistic uncertainty on a softmax classifier is insufficient for robust zero-day detection*, motivating the extension in Section III.

III. PROBLEM STATEMENT

A. Motivation and Research Gap

The 70% evasion rate reveals that the latent feature clusters are too broad and overlapping. A rogue signal landing near a legitimate cluster produces low uncertainty despite being from an unseen device. True zero-day security requires geometrically tightening intra-class clusters so that any unknown signal falls measurably outside all known device regions.

B. Problem Statement

Phase 1—FGSM Adversarial Fine-Tuning. Input gradients are used to craft adversarial perturbations:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{J}(\theta, \mathbf{x}, y)), \quad \epsilon = 0.05. \quad (4)$$

The pre-trained model is fine-tuned for 5 epochs on 3,000 samples ($lr = 10^{-4}$), backpropagating the adversarial loss $\mathcal{L}(\mathbf{x}_{\text{adv}}, y)$ separately from the clean-signal pass to avoid gradient double-counting. Total loss decreased from 10.01 to 5.20; clean accuracy dropped from $\sim 51\%$ to $\sim 31\%$, reflecting the adversarial robustness trade-off.

Phase 2—1-NN Feature-Space Anomaly Detection. The softmax heads are discarded. Raw 64-dim feature vectors $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^{64}$ are extracted after the attention and pooling layers. A reference database of $N = 100$ legitimate feature vectors is built offline. The anomaly score for any test signal is:

$$d^* = \min_{i \in \mathcal{D}} \|\mathbf{z}^* - \mathbf{z}_i\|_2. \quad (5)$$

Signals with $d^* > \tau$ are classified as rogue.

Results and Discussions. Fig. 2 shows the 1-NN distance histogram after adversarial hardening. Legitimate signals (blue) cluster at distances below 0.75, while rogue signals (red) are geometrically expelled to higher distances. The threshold $\tau = 0.80$ perfectly separates both populations with no overlap between the two distributions. Fig. 3 shows the t-SNE projection of the 64-dim feature space: known device clusters (coloured circles) are compact and well-separated, while zero-day rogue signals (red $\times$) are distributed at the periphery, confirming their geometric displacement by adversarial fine-tuning.

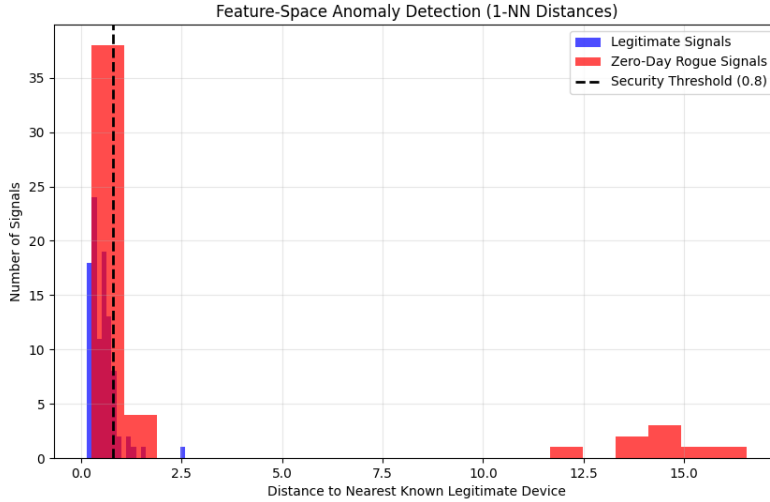


Fig. 2. 1-NN distance histogram after adversarial hardening. Legitimate signals (blue) concentrate near zero; rogue signals (red) exhibit higher distances. Dashed line: security threshold $\tau = 0.80$.

IV. CONCLUSIONS

Standard MC Dropout uncertainty on a closed-set softmax classifier is insufficient for zero-day physical-layer security: 70% of rogue signals evaded detection by falling within broad,

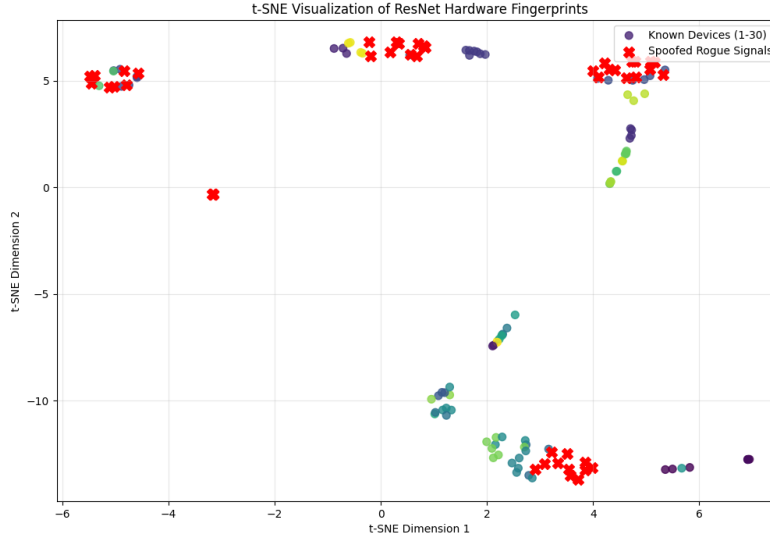


Fig. 3. t-SNE projection of 64-dim feature vectors. Known device clusters (coloured circles) are compact. Zero-day rogue signals (red $\times$) are scattered at the periphery, confirming geometric separation by the hardened model.

overlapping legitimate feature clusters. FGSM adversarial fine-tuning combined with a 1-NN feature-space anomaly detector restructures the latent geometry to produce clear distributional separation, as confirmed by the distance histogram and t-SNE visualisation. The key insight is that *classification accuracy is the wrong objective for open-set security*—geometric control over the latent space is what matters. Future work should evaluate Mahalanobis distance metrics and robustness under real-world multipath channel conditions.

V. APPENDIX

A. Adversarial Training (*train_adv.py*)

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader
5 from dataset import LoRaDataset
6 from models.probabilistic import ProbabilisticMultiTaskResNet
7 from tqdm import tqdm
8 import os
9 import sys
10
11 def fgsm_attack(signal, epsilon, data_grad):

```

```

12     """
13     The Fast Gradient Sign Method (FGSM).
14     Extracts the gradient of the signal and applies mathematical noise
15     in the exact direction that maximizes the model's error.
16     """
17     sign_data_grad = data_grad.sign()
18     perturbed_signal = signal + epsilon * sign_data_grad
19     return perturbed_signal
20
21 def train_adversarial():
22     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
23     print(f"--- INITIATING ADVERSARIAL TRAINING (FGSM) ON {str(device).upper()} ---")
24
25     train_dataset = LoRaDataset(r'data\raw\Train\dataset_training_no_aug.h5',
26                                subset_size=3000)
27     train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
28
29     model = ProbabilisticMultiTaskResNet(num_devices=30).to(device)
30     model.load_state_dict(torch.load("lora_model_drop30.pth", map_location=device,
31                                     weights_only=True))
32
33     criterion = nn.CrossEntropyLoss()
34     optimizer = optim.Adam(model.parameters(), lr=0.0001)
35
36     num_epochs = 5
37     epsilon = 0.05
38
39     for epoch in range(num_epochs):
40         model.train()
41         running_loss = 0.0
42         correct_clean = 0
43         correct_adv = 0
44         total = 0
45         progress_bar = tqdm(train_loader, desc=f"Adv. Epoch {epoch+1}/{num_epochs}")
46
47         for x, y_dev, _ in progress_bar:
48             x, y_dev = x.to(device), y_dev.to(device)
49             x.requires_grad = True
50
51             # Forward pass on clean signal
52             model.zero_grad()
53             out_clean, _ = model(x)

```

```

52     loss_clean = criterion(out_clean, y_dev)
53     loss_clean.backward(retain_graph=True)
54     data_grad = x.grad.data
55
56     # Generate adversarial signal and forward pass
57     x_adv = fgsm_attack(x, epsilon, data_grad)
58     out_adv, _ = model(x_adv)
59     loss_adv = criterion(out_adv, y_dev)
60
61     # Train on adversarial signal
62     optimizer.zero_grad()
63     loss_adv.backward()
64     optimizer.step()
65
66     running_loss += (loss_clean.item() + loss_adv.item())
67     _, pred_clean = torch.max(out_clean.data, 1)
68     _, pred_adv = torch.max(out_adv.data, 1)
69     total += y_dev.size(0)
70     correct_clean += (pred_clean == y_dev).sum().item()
71     correct_adv += (pred_adv == y_dev).sum().item()
72     progress_bar.set_postfix({
73         'Loss': f"{running_loss/len(train_loader):.4f}",
74         'Clean_Acc': f"{100.*correct_clean/total:.1f}%",
75         'Adv_Acc': f"{100.*correct_adv/total:.1f}%"
76     })
77
78     torch.save(model.state_dict(), "lora_model_hardened.pth")
79     print("\nAdversarial Training Complete. Model saved to lora_model_hardened.pth")
80
81 if __name__ == "__main__":
82     train_adversarial()

```

B. 1-NN Anomaly Detection (test_anomaly.py)

```

1 import torch
2 import torch.nn.functional as F
3 from torch.utils.data import DataLoader
4 from dataset import LoRaDataset
5 from models.probabilistic import ProbabilisticMultiTaskResNet
6 import os
7 import sys

```



```

8
9 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
10 from config import KNN_ANOMALY_THRESHOLD
11
12 def extract_features(model, x):
13     x = x.view(-1, 1, 128, 128)
14     x = F.relu(model.bn1(model.conv1(x)))
15     x = model.layer1(x)
16     x = model.layer2(x)
17     x = model.layer3(x)
18     x = model.attention(x)
19     x = model.pool(x)
20     features = x.view(x.size(0), -1)
21     return features
22
23 def test_distance_anomaly():
24     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
25     print(f"--- INITIATING FEATURE-SPACE ANOMALY DETECTION ON {str(device).upper()}
26           ---")
27
28     model = ProbabilisticMultiTaskResNet(num_devices=30).to(device)
29     model.load_state_dict(torch.load("lora_model_drop30.pth", map_location=device,
30     weights_only=True))
31     model.eval()
32
33     # Load 105 samples total to prevent data leaks: 100 for DB, 5 for testing
34     print("Building database from legitimate devices...")
35     legit_dataset = LoRaDataset(r'data\raw\Train\dataset_training_no_aug.h5',
36     subset_size=105)
37     legit_loader = DataLoader(legit_dataset, batch_size=105, shuffle=True)
38     legit_batch_x, _, _ = next(iter(legit_loader))
39     db_x = legit_batch_x[:100]
40     legit_test_x = legit_batch_x[100:]
41
42     with torch.no_grad():
43         legit_database = extract_features(model, db_x.to(device))
44
45     rogue_file_path = r'data\raw\Test\dataset_rogue.h5'
46     if not os.path.exists(rogue_file_path):
47         print("Error: Rogue dataset not found!")
48         return

```

```

47     print("\nEvaluating Rogue Signals...")
48     rogue_dataset = LoRaDataset(rogue_file_path, subset_size=10, is_rogue_dataset=True
49                                )
50     rogue_loader = DataLoader(rogue_dataset, batch_size=10, shuffle=True)
51     rogue_x, _, _ = next(iter(rogue_loader))
52
53     # 5 guaranteed-new legitimate samples followed by 10 rogue samples
54     test_signals = torch.cat((legit_test_x, rogue_x), dim=0).to(device)
55
56     with torch.no_grad():
57         test_features = extract_features(model, test_signals)
58         distances_matrix = torch.cdist(test_features, legit_database)
59         min_distances, _ = torch.min(distances_matrix, dim=1)
60
61     print("\n--- NEAREST NEIGHBOR ANOMALY REPORT ---")
62     for i, dist in enumerate(min_distances):
63         val = dist.item()
64         true_type = "LEGITIMATE" if i < 5 else "ROGUE"
65         if val > KNN_ANOMALY_THRESHOLD:
66             status = "SECURITY ALERT: Hardware Fingerprint Unrecognized!"
67         else:
68             status = "Authenticated"
69         print(f"Sample {i+1} ({true_type}) | Distance: {val:.4f} | {status}")
70
71 if __name__ == "__main__":
72     test_distance_anomaly()

```

C. Visualisation (*visualize.py*)

```

1  import torch
2  import torch.nn.functional as F
3  from torch.utils.data import DataLoader
4  from dataset import LoRaDataset
5  from models.probabilistic import ProbabilisticMultiTaskResNet
6  import matplotlib.pyplot as plt
7  from sklearn.manifold import TSNE
8  import numpy as np
9  import os
10 import sys
11 import warnings
12

```

```

13 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
14 from config import KNN_ANOMALY_THRESHOLD
15
16 warnings.filterwarnings('ignore')
17
18 def extract_features(model, x):
19     x = x.view(-1, 1, 128, 128)
20     x = F.relu(model.bn1(model.conv1(x)))
21     x = model.layer1(x)
22     x = model.layer2(x)
23     x = model.layer3(x)
24     x = model.attention(x)
25     x = model.pool(x)
26     return x.view(x.size(0), -1)
27
28 def generate_plots():
29     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
30     print(f"--- GENERATING VISUALIZATIONS ON {str(device).upper()} ---")
31
32     model = ProbabilisticMultiTaskResNet(num_devices=30).to(device)
33     model.load_state_dict(torch.load("lora_model_hardened.pth", map_location=device,
34                                     weights_only=True))
35     model.eval()
36
37     print("Extracting Legitimate Features...")
38     legit_dataset = LoRaDataset(r'data\raw\Train\dataset_training_no_aug.h5',
39                                subset_size=100)
40     legit_loader = DataLoader(legit_dataset, batch_size=100, shuffle=True)
41     legit_x, legit_y, _ = next(iter(legit_loader))
42     with torch.no_grad():
43         legit_features = extract_features(model, legit_x.to(device))
44
45     print("Extracting Rogue Features...")
46     rogue_dataset = LoRaDataset(r'data\raw\Test\dataset_rogue.h5', subset_size=50,
47                                is_rogue_dataset=True)
48     rogue_loader = DataLoader(rogue_dataset, batch_size=50, shuffle=True)
49     rogue_x, _, _ = next(iter(rogue_loader))
50     with torch.no_grad():
51         rogue_features = extract_features(model, rogue_x.to(device))
52
53     print("Generating Distance Histogram...")
54     distances_matrix = torch.cdist(rogue_features, legit_features)

```

```

52     rogue_min_distances, _ = torch.min(distances_matrix, dim=1)
53     legit_internal = torch.cdist(legit_features, legit_features)
54     legit_internal.fill_diagonal_(float('inf'))
55     legit_min_distances, _ = torch.min(legit_internal, dim=1)
56
57     plt.figure(figsize=(10, 6))
58     plt.hist(legit_min_distances.cpu().numpy(), bins=20, alpha=0.7, color='blue',
59             label='Legitimate Signals')
60     plt.hist(rogue_min_distances.cpu().numpy(), bins=20, alpha=0.7, color='red', label
61             = 'Zero-Day Rogue Signals')
62     plt.axvline(x=KNN_ANOMALY_THRESHOLD, color='black', linestyle='dashed',
63                linewidth=2, label=f'Security Threshold ({KNN_ANOMALY_THRESHOLD})')
64     plt.title('Feature-Space Anomaly Detection (1-NN Distances)')
65     plt.xlabel('Distance to Nearest Known Legitimate Device')
66     plt.ylabel('Number of Signals')
67     plt.legend()
68     plt.grid(True, alpha=0.3)
69     plt.savefig('distance_histogram.png')
70     print("> Saved 'distance_histogram.png'")
71
72     print("Generating t-SNE Scatter Plot...")
73     all_features = torch.cat((legit_features, rogue_features), dim=0).cpu().numpy()
74     legit_labels = legit_y.cpu().numpy()
75     tsne = TSNE(n_components=2, perplexity=30, random_state=42)
76     features_2d = tsne.fit_transform(all_features)
77
78     plt.figure(figsize=(12, 8))
79     plt.scatter(features_2d[:100, 0], features_2d[:100, 1],
80                c=legit_labels, cmap='viridis', marker='o', s=50, alpha=0.8,
81                label='Known Devices (1-30)')
82     plt.scatter(features_2d[100:, 0], features_2d[100:, 1],
83                c='red', marker='X', s=100, label='Spoofed Rogue Signals')
84     plt.title('t-SNE Visualization of ResNet Hardware Fingerprints')
85     plt.xlabel('t-SNE Dimension 1')
86     plt.ylabel('t-SNE Dimension 2')
87     plt.legend()
88     plt.grid(True, alpha=0.3)
89     plt.savefig('tsne_features.png')
90     print("> Saved 'tsne_features.png'")
91
92 if __name__ == "__main__":
93     generate_plots()

```

REFERENCES

- [1] G. Shen, J. Zhang, A. Marshall, and J. R. Cavallaro, "Towards scalable and channel-robust radio frequency fingerprint identification for LoRa," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 774–787, 2022.
- [2] Y. E. Sagduyu and T. Erpek, "Adversarial attack and defense for LoRa device identification and authentication via deep learning," 2024.